In-House Legal Intelligence System

CS-781 Capstone: what the project is, and how Phase 1 is built

Manish Kumar (251330604)

CS-781, IIT Kanpur

Phase 1: 23 September – 6 October 2026

Outline

1. The capstone
2. Phase 1: the task
3. Phase 1: our pipeline
4. Phase 1: the four architectures
5. Phase 1: training, decoding, evaluation
6. Running the code

The brief: a law firm that cannot use cloud LLMs

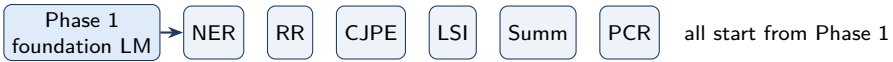
- You are the new in-house AI developer at a law firm.
- Client matters and privileged material cannot leave the firm's infrastructure. Sending them to a third-party LLM API is ruled out.
- The partners want a **self-hosted legal language foundation**. It must run entirely on hardware they control.
- It must then support the legal NLP tasks the firm needs every day.
- A rival engineer builds against the same brief, data and rules. The competition runs on a public Codabench leaderboard.

What "success" means

- Not one lucky high score.
- A system you understand well enough to **defend, extend and fix**.
- Honest experiments: what worked, what failed, and why.
- Defended in a viva, as you would defend it to a sceptical partner.

Capstone at a glance: seven phases, one foundation model

#	Task	Input	Output	Primary metric
1	Legal LM pre-training	512-word legal prompt	128-word continuation	BERTScore F1
2	Legal NER (L-NER)	legal document	character spans + labels	strict Macro-F1
3	Rhetorical Role (RR)	document sentences	one role per sentence	Macro-F1
4	Judgment Prediction (CJPE)	case text	binary 0/1	Macro-F1
5	Statute Identification (LSI)	legal document	one or more statute IDs	Macro-F1
6	Abstractive Summarisation	legal document	generated summary	BERTScore F1
7	Prior Case Retrieval (PCR)	query + candidate pool	top-7 ranked IDs	Micro-F1@7
8	TBA (e.g. QA on your model)	TBA	TBA	TBA



Timeline (IST)

Milestone	Dates (2026)
Codabench registration	26 Sep, 12 noon
Phase 1: LM pre-training	**23 Sep – 6 Oct**
Phase 2: L-NER	7 – 11 Oct
Phase 3: RR	12 – 16 Oct
Phase 4: CJPE	17 – 21 Oct
Phase 5: LSI	22 – 26 Oct
Phase 6: Summarisation	27 Oct – 2 Nov
Phase 7: PCR	3 – 7 Nov
Report + code	8 Nov
Viva	9 – 13 Nov (tentative)

Deadlines are strict

Codabench needs 15–30 minutes per submission, and longer under load. Submit early in each window. Delay excuses are not accepted.

Submission counts

- Phase 1: **at least 4**. Phases 2–7: **at least 6**.
- Max 40 per phase and 10 per day.
- Identical re-submissions do not count.

Data, access and the submission format

Dataset

`Exploration-Lab/CS-781-Capstone` on Hugging Face (gated: accept the terms, then log in). There is one folder per phase, `Phase_1_Pretraining` to `Phase_7_PCR`. Test references are private.

Only official data

No external legal corpora, extra judgments, statutes or scraped court data.

Same packaging in every phase

`submission.zip`

`prediction.json` at the root

not `submission.zip/folder/prediction.json`

- Include **every** official test ID.
- Add **no** unknown ID. The evaluator checks both.

Phases 2–7: shared rules for adaptation

- Start from **your own Phase-1 models**. Other pretrained models and LLM APIs are prohibited.
- Experiment with **all four** Phase-1 architectures. Fine-tuning a single model is not enough.
- At least **6 substantially different** experiments per phase:
  - ≥ 5 fine-tuning / PEFT techniques, and LORMA is compulsory;
  - ≥ 1 knowledge distillation (teacher $\rightarrow$ smaller student).
- Changing only hyperparameters does **not** make a new experiment.

Technique menu (varies by phase)

Full and partial fine-tuning, layer freezing, LoRA, QLoRA, AdaLoRA, rsLoRA, **LORMA**, Prefix tuning, Prompt tuning, Houlsby/Pfeiffer adapters, BitFit, IA^3, knowledge distillation.

Why Phase 1 matters so much

All four Phase-1 models are reused six more times. Their tokenizer, context length and architecture shape every later phase.

Phases 2 and 3: extraction and document structure

Phase 2: Legal NER

- Output `[start, end, label]` character spans per document.
- 12 labels: APP, RESP, A.COUNSEL, R.COUNSEL, JUDGE, WIT, AUTH, COURT, STAT, PREC, DATE, CASENO.
- **Strict** matching. An offset that is off by one character counts as wrong.
- Key issues: character-to-token alignment, boundary errors, boilerplate false positives, class imbalance.

Phase 3: Rhetorical roles

- Output exactly one label per sentence, in order.
- 13 labels, e.g. Fact, Issue, ArgumentPetitioner, Statute, PrecedentReliedUpon, RatioOfTheDecision, RulingByPresentCourt, None.
- Two domains: Corporate Law (CL) and IT.
- Key issues: sentence versus document context, and confusable roles.

Phases 4 and 5: decisions and statutes

Phase 4: Court judgment prediction

- Predict a binary outcome: `{"id":..., "pred": 0|1}`.
- Judgments are **long**. Options include chunking, sliding windows, hierarchical or attention pooling, and section selection.
- Scored by Macro-F1, so both classes matter. Handle class imbalance.

Phase 5: Legal statute identification

- **Multi-label**: output a set of statute IDs per document.
- Needs multi-label loss, class weights and rare-statute handling.
- **Thresholds** can be global or per label, with calibration.
- Macro-F1 rewards getting rare statutes right.

Phases 6 and 7: generation and retrieval

Phase 6: Abstractive summarisation

- Generate a concise summary of a long legal document.
- Scored by BERTScore F1, with ROUGE-1 and ROUGE-L as secondary metrics.
- Needs hierarchical or chunk-then-summarise strategies, length control and redundancy control.
- Phase-1 generation quality carries over directly.

Phase 7: Prior case retrieval

- Rank 7 unique candidate IDs per query, taken from that split's candidate pool.
- Scored by Micro-F1@7 against the full gold set.
- Options include contrastive or pairwise objectives, hard negatives, pooling and chunked representations.
- The student model in distillation learns from teacher similarity scores.

How the project is graded, and the integrity rules

Leaderboard is only one criterion

- Data analysis and preprocessing
- Minimum submissions and real experimentation
- Code quality, documentation and the GitHub repository
- Analysis of what worked and failed, and why
- Novelty within the compute limits
- Private-test performance and rank
- Viva performance and report quality

Rules

- Individual work. Similar submissions can lead to an **F grade**.
- LLMs **may** help with ancillary code (plots) and with understanding architectures.
- LLMs **may not** write the model architecture, tune hyperparameters, or analyse results.
- The LLM declaration is mandatory. Cite every paper and snippet used.
- Compute budget: free Colab or Kaggle GPUs. Checkpoint often.

Final deliverables

```
CS781-CP-First-Last-Roll-report_code.zip
|-- code/
|   |-- phase1/
|   |-- phase2/  ...
`-- report/
    `-- CS781-CP-...-report.pdf
```

- No extra wrapper folder.
- Submitted on Hello IITK.
- Include a GitHub repository with a detailed README for each phase and submission.

The report (LaTeX) must explain

1. the problem and the data used;
2. the baselines tried and the final approach;
3. the experiments that led there;
4. what failed and why;
5. what the results show, the remaining limitations, and the next steps.

It should be reasoning, not a gallery of leaderboard screenshots.

Phase 1: continue legal text

Input: a 512-word legal prompt.

```
{ "1": {"text": "The issue involved in
         these petitions is ..."},
  "2": {"text": "In ..."} }
```

Output: a 128-word continuation for each ID.

```
{ "1": {"text": "<128 generated words>"},
  "2": {"text": "..."} }
```

The output must contain only the continuation, never the prompt.

Metrics

- **Primary:** BERTScore F1, which measures semantic similarity to the hidden true continuation.
- **Secondary:** ROUGE-L F1 (longest common subsequence) and BLEU-4 (n-gram precision).

The model is the real product

The Phase-1 model is the **foundation** for Phases 2–7. Its tokenizer, context length and quality are inherited everywhere.

Phase 1 rules

Must

- Train **from scratch** on the provided corpus.
- Stay within **50M – 300M parameters**.
- Implement and train your own architecture.
- Stay practical on free Colab or Kaggle GPUs.
- Explore **at least four substantially different architectures**.

Not allowed

- Pretrained language models or pretrained weights (HF, fairseq, . . .).
- External legal corpora or datasets.
- Closed commercial APIs for training, inference or evaluation.
- Any foundation model above 300M parameters.

Allowed

The course BPE tokenizer or a new tokenizer, static embeddings (Word2Vec, GloVe), and any amount of the provided corpus.

Different means different

Changing width, depth, heads or learning rate of the *same* design does **not** count as a new architecture.

Phase 1 data

File (`Phase_1_Pretraining/`)	Size
`Train/train_acts.json`	0.73 GB
`Train/train_supreme_court.json`	1.32 GB
`Train/train_tribunals.json`	6.48 GB
`Train/train_high_courts.json`	10.67 GB
`Test/test.json`	3 MB

- More than **4.5 billion tokens** in total.
- No development split is provided. You build your own.
- The starter notebook uses about 200M training and 5M validation tokens, a GPT-2 tokenizer, a 2048 context, and 8 layers $\times$ 8 heads $\times$ 560 dimensions.

Consequences for the design

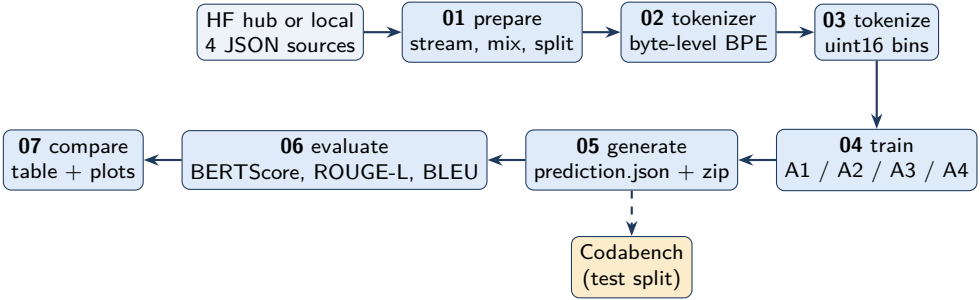
- The files cannot be loaded into RAM, so they must be **streamed**.
- Compute, not data, is the limit. Choosing *which* tokens to use is an experiment in itself.
- The four sources differ in style. Acts are statutory text; the others are judgments.
- The test prompts cannot be scored locally, so we build a **dev set** in the same format.

What the experiments should cover

- **Tokenizer:** BPE versus others, vocabulary size, tokens per word.
- **Data mixing:** which sources, and in what ratios.
- **Architecture:** attention, feed-forward, normalisation and positional scheme.
- **Context length:** long-range legal dependencies versus compute.
- **Training:** learning-rate schedule, warm-up, batch size, accumulation and duration.
- **Regularisation:** dropout and weight decay.
- **Validation:** split construction, early stopping and checkpoint choice.
- **Decoding:** temperature, top-k, top-p and repetition control.
- **Compression:** knowledge distillation into a compact model.
- **Key question:** how does validation perplexity relate to generation quality?

"A weaker model with clear evidence and thoughtful analysis is more valuable than an unexplained lucky submission."

End-to-end pipeline: seven small scripts



- One YAML file per architecture *inherits* `base.yaml`. All four share data, tokenizer, budget and schedule, which makes the comparison **controlled**.
- Any value can be changed from the command line, e.g. `-set train.lr=3e-4 run_name=A1_lr3e-4`.
- Every run saves its resolved config, a metrics log, a summary, and best and last checkpoints.

Step 01: streaming, mixing and splitting

- Files are read **record by record** with `ijson`, over HTTP from the hub. Nothing is fully downloaded.
- The JSON layout is auto-detected: array, JSON Lines, or ID-keyed object.
- Source s with ratio r_s receives a budget of $B \cdot r_s$ tokens. The default is $B = 200$M with Acts 0.10, SC 0.30, HC 0.35 and Tribunals 0.25.
- Documents are skipped at random (`doc_keep_prob`) so reads spread across each file.
- The train/val split is made **by document**, so no validation text leaks into training.

Local dev set = a copy of the test format

From each held-out document we take a random window of 640 words:

$$\underbrace{w_1 \ldots w_{512}}_{\texttt{dev_prompts.json}} \quad \underbrace{w_{513} \ldots w_{640}}_{\texttt{dev_references.json}}$$

This lets us compute BERTScore, ROUGE-L and BLEU-4 locally before spending a Codabench submission.

Step 02: a legal byte-level BPE tokenizer

- BPE starts from the 256 bytes and **repeatedly merges the most frequent adjacent pair** until it reaches the target vocabulary size (default 32k).
- Byte-level means there are **no unknown tokens**: Devanagari names, "§", and OCR noise can all be encoded.
- Trained on legal text, frequent words such as *appellant*, *respondent* and *Section* become single tokens.
- Special tokens are <|endoftext|> for document boundaries and <|pad|>.

Efficiency metric: fertility

$$\text{fertility} = \frac{\#\text{tokens}}{\#\text{words}}$$

Lower fertility means shorter sequences. More of the 512-word prompt then fits in the context, and training covers more text per step.

Experiment

Compare legal BPE at 16k, 32k and 48k against GPT-2's 50k vocabulary. Measure tokens per word, validation loss, and the parameters spent on the embedding table.

Step 03 and the language-modelling objective

Token bins. All documents are joined with `<|endoftext|>` into one flat `uint16` file. Training reads random windows through a memory map, so RAM use stays tiny.

Causal LM. For a sequence $x_1 \dots x_T$:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}), \qquad \text{PPL} = e^{\mathcal{L}}$$

Continuation loss: one metric for all four models

With a prefix of length p as context, score only the tokens after it:

$$\mathcal{L}_{\text{cont}} = -\frac{1}{T-p}\sum_{t=p+1}^{T} \log p_\theta(x_t \mid x_{\le p},\, x_{p+1..t-1})$$

Validation uses $p = 768$ and scores the next 257 tokens for *every* architecture. This mirrors the task of reading the prompt and then writing.

One interface, four objectives: the `prefix_len` trick

Every model implements `model(idx, p)`, returning logits that predict `idx[:, p:]`.

Model	Training prefix length	What the prefix does
A1 GPT	always 1, so every token is supervised	read causally
A2 Enc–dec	random in $[L/4,\ L-64]$	encoder reads it, decoder writes
A3 Prefix-LM	1 with probability 0.5, else random in $[64,\ 0.75L]$	read **bidirectionally** in the same stack
A4 Efficient	always 1	read causally (local + global)

Why it helps

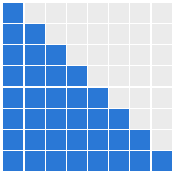
One trainer, evaluator and generation loop serve all four models.

Trade-off to analyse

A2 and A3 do not score the prefix, so fewer tokens are supervised. Do they still win?

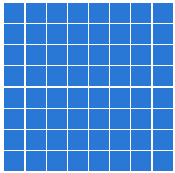
Attention masks: the core difference

GPT (A1), decoder self-attn (A2)



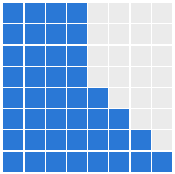
A1 causal

bidirectional over prompt



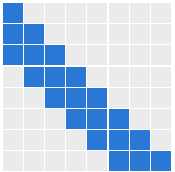
A2 encoder

bidirectional prompt + causal rest



A3 prefix-LM ($p = 4$)

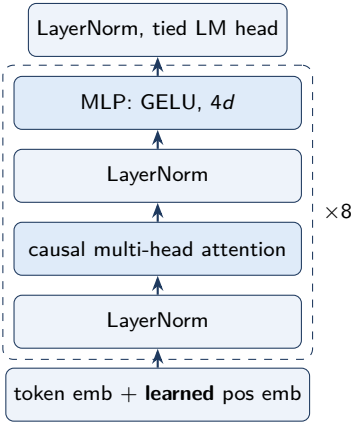
sliding window (some layers)



A4 local, $W = 3$

Rows are queries and columns are keys. A blue cell means the query may attend to that key. A4 uses the local mask in most layers and the full causal mask in every third layer.

A1: decoder-only Transformer (GPT-2 style), the baseline

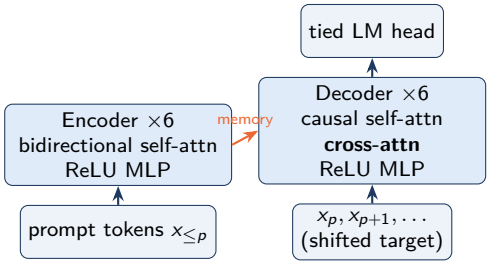


- Pre-LN residual block: $x \leftarrow x + \mathrm{Attn}(\mathrm{LN}(x))$, then $x \leftarrow x + \mathrm{MLP}(\mathrm{LN}(x))$.
- Attention: $\mathrm{softmax}\big(QK^\top/\sqrt{d_h} + M_{\text{causal}}\big)\,V$.
- Positions come from a learned table of 1024 rows, so the model cannot read beyond 1024 tokens.
- The output head is tied to the embedding matrix, which saves $V \cdot d$ parameters.
- Uses $d = 768$, 8 layers and 12 heads, for **82.1M** parameters.

Role in the study

The reference point, closest to the starter notebook. Every other design is measured against it.

A2: encoder–decoder Transformer



- Each block is cut at a random point p. The encoder reads $[0, p)$ and the decoder predicts $[p, L)$.
- The decoder input starts with the *last prompt token*, so no BOS token is needed.
- Fixed **sinusoidal** positions, with embeddings scaled by $\sqrt{d}$ as in Vaswani et al.
- Uses $d = 640$ and 6+6 layers, for **89.4M** parameters.

Hypothesis

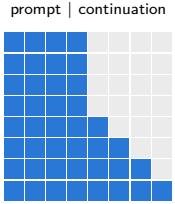
The objective matches the task: read the prompt, then write.

A3: hybrid prefix-LM with ALiBi (UniLM / GLM style)

- One stack is an encoder for the prompt and a decoder for the continuation. Only the mask changes, and there is no cross-attention.
- There are **no position embeddings**. ALiBi adds a distance penalty per head h:

$$\text{score}_{ij} = \frac{q_i^\top k_j}{\sqrt{d_h}} - m_h\,|i-j|, \quad m_h = 2^{-8h/H}$$

- ALiBi can extrapolate to longer inputs than seen in training.
- Half the examples are plain causal; the rest use a random prefix. 8 layers, $d = 768$, **81.3M** parameters.



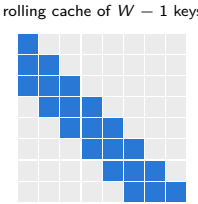
mask with $p = 4$

Hypothesis

Bidirectional reading of the prompt gives a richer context representation than A1, without A2's extra encoder parameters.

A4: efficient decoder (LLaMA, Mistral and Gemma-2 ideas)

- **Grouped-query attention**: 12 query heads share 4 KV heads, so the KV-cache is 3× smaller.
- **Sliding window** $W = 256$ costs $O(TW)$, not $O(T^2)$. Every third layer is global.
- **RoPE** rotates q and k by angle $\theta_i \cdot \text{pos}$, so scores depend on relative position.
- **RMSNorm** $x/\sqrt{\frac{1}{d}\sum x_i^2 + \epsilon} \odot g$ and **SwiGLU** $W_2\big(\text{SiLU}(W_g x) \odot W_u x\big)$, with no biases.
- 10 layers, $d = 768$, **87.5M** parameters.



local layer, $W = 3$

Hypothesis

Better quality per FLOP. Faster generation, and a smaller memory footprint for long legal documents in Phases 4–7.

The four designs side by side

	A1 GPT	**A2 Enc–Dec**	**A3 Prefix-LM**	**A4 Efficient**
Stacks	1 decoder	encoder + decoder	1 shared	1 decoder
Prompt attention	causal	bidirectional	bidirectional	causal, local/global
Positions	learned absolute	sinusoidal	ALiBi (relative)	RoPE (relative)
Norm	LayerNorm	LayerNorm	LayerNorm	RMSNorm
MLP	GELU	ReLU	GELU	SwiGLU
KV heads	12	10	12	**4** (GQA)
Extra	–	cross-attention	prefix mask	sliding window
Objective	causal LM	prefix → continuation	mixed causal/prefix	causal LM
Parameters	82.1M	89.4M	81.3M	87.5M

All four use the 32k legal BPE, a 1024-token context, and a tied output head. Parameter counts are matched to within 10%, so differences come from **design**, not size.

Where the parameters go

For a standard block with width d and MLP width $4d$:

$$\underbrace{4d^2}_{Q,K,V,O} + \underbrace{8d^2}_{\text{MLP}} = 12d^2$$

For $d = 768$ that is 7.1M per layer, or 56.6M for 8 layers.

The embedding table adds $V \cdot d = 32{,}064 \times 768 = 24.6\text{M}$.

Total: about **82M**, inside the 50–300M window.

Levers the rules leave open

- The vocabulary size trades embedding parameters against fertility.
- GQA and SwiGLU re-spend parameters: $d_{ff} = 2048$ with 3 matrices costs about the same as a $4d$ GELU MLP.
- There is room to scale to 150–300M if the GPU time allows. Compare the gain against the extra time per step.

Step 04: one training loop for every architecture

- **AdamW** with $\beta = (0.9, 0.95)$ and weight decay 0.1 on matrices only.
- **Linear warm-up then cosine decay**:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}\big(\eta_{\max} - \eta_{\min}\big)\big(1 + \cos \pi \tfrac{t - t_w}{T - t_w}\big)$$

- Batch: 8×8 accumulation $\times$ 1024 $=$ 65k tokens per step. 3000 steps is about 197M tokens.
- **Mixed precision**: bf16, or fp16 with a GradScaler on a T4. Gradients are clipped at 1.0.
- Validation every 250 steps on fixed blocks; the best checkpoint is kept.

Built for free GPUs

- `last.pt` includes the optimizer state and is written atomically.
- Rerunning the command **resumes** automatically.
- Point `paths.runs_dir` at Google Drive.

Logged for the report

Losses, perplexity, learning rate, gradient norm, tokens/s and peak memory.

Step 05: decoding exactly 128 words

At each step $p(v) \propto \exp(z_v/\tau)$, then:

- **top-k** keeps the k most likely tokens;
- **top-p** keeps the smallest set whose probability mass reaches p;
- **repetition penalty** divides positive logits (or multiplies negative ones) by r for recently used tokens;
- $\tau = 0$ gives greedy decoding.

Defaults are $\tau = 0.8$, $k = 50$, $p = 0.95$ and $r = 1.1$. These are an *experiment*, not an answer.

Engineering details

- A KV-cache handles prefill, then one token per step.
- Prompts are sorted by length and left-truncated per batch, so no padding is needed.
- The end-of-text token is **banned**, so the model never stops early.
- Generation stops at 129 complete words and keeps the first 128.
- Only the continuation is written. The IDs and format are checked before zipping.

Step 06: the metrics, and what they reward

BERTScore (primary) matches every token to its most similar token by contextual embedding:

$$R = \frac{1}{|r|} \sum_{r_i} \max_{h_j} \cos(\mathbf{r}_i, \mathbf{h}_j)$$

$$P = \frac{1}{|h|} \sum_{h_j} \max_{r_i} \cos(\mathbf{r}_i, \mathbf{h}_j), \qquad F_1 = \frac{2PR}{P+R}$$

It rewards the *right content and vocabulary*, even when the wording differs.

ROUGE-L is the F-measure of the longest common subsequence. It rewards the right words in the right order.

BLEU-4 is the geometric mean of 1–4-gram precisions times a brevity penalty. It rewards exact phrases.

Open question to study

Lower perplexity does not guarantee higher BERTScore. Decoding temperature can move BERTScore more than a small change in loss.

Compression: knowledge distillation

A trained teacher (for example A1 at 82M) guides a smaller student (an A4 variant at 47M):

$$\mathcal{L} = \alpha\,\mathcal{L}_{\text{CE}} + (1-\alpha)\,\tau^2\,\mathrm{KL}\big(\sigma(z^T/\tau)\,\|\,\sigma(z^S/\tau)\big)$$

- Softened teacher probabilities ($\tau = 2$) carry "dark knowledge", such as which wrong tokens are nearly right.
- The shared `prefix_len` interface lets the teacher and student even be different architectures.
- This is enabled with one config file: `distill_efficient_small.yaml`.

Questions to answer

- How much of the teacher's continuation loss and BERTScore does the student keep?
- How do its speed and memory compare?
- Is distillation better than training the same small model without a teacher?

Also a rehearsal

Distillation is **mandatory** in Phases 2–7.

A controlled experiment plan

Question	Controlled change (everything else fixed)	Read out
Which architecture?	A1 vs A2 vs A3 vs A4	val loss, BERTScore, tok/s
Which tokenizer?	legal BPE 16k / 32k / 48k vs GPT-2	fertility, val loss
Which data?	drop one source; Acts-heavy vs judgment-heavy mix	val loss by source
How much context?	block size 512 / 1024 / 2048	val loss, memory, speed
How to optimise?	learning rate, warm-up, batch size, steps	loss curves
Overfitting?	dropout, weight decay, early stopping	train–val gap
Which checkpoint?	best.pt vs last.pt vs intermediate	dev BERTScore
How to decode?	greedy vs sampling; τ, top-p, repetition penalty	dev BERTScore
Compression?	distilled student vs same student from scratch	quality vs size

Each row is a `-set` override. The run's config and metrics are saved automatically, and `07_compare.py` builds the table and plot. Use the local dev set to choose which runs are worth one of the Codabench submissions.

How to run it

```
pip install -r requirements.txt
huggingface-cli login                        # the dataset is gated

bash run_smoke_test.sh                       # whole pipeline on toy data, minutes

python scripts/01_prepare_data.py    --config configs/base.yaml
python scripts/02_train_tokenizer.py --config configs/base.yaml --compare-gpt2
python scripts/03_tokenize.py        --config configs/base.yaml
python scripts/04_train.py    --config configs/a1_gpt.yaml   # a2_encdec, a3_prefixlm, a4_efficient
python scripts/05_generate.py --run runs/A1_gpt --split dev
python scripts/06_evaluate.py --pred runs/A1_gpt/predictions/dev_.../prediction.json
python scripts/05_generate.py --run runs/A1_gpt --split test  # -> submission.zip
python scripts/07_compare.py                                   # table + plot
```

Or run `bash run_all.sh` for everything, with all four architectures.

Code organisation

```
legal_lm/
  config.py            YAML + inherit + --set
  data/corpus.py       stream, mix, split, dev set
  data/tokenizer.py    byte-level BPE
  data/dataset.py      uint16 bins, batches
  models/layers.py     attention, RoPE, ALiBi,
                       RMSNorm, SwiGLU, masks
  models/base.py       common interface
  models/decoder_gpt.py      A1
  models/encoder_decoder.py  A2
  models/prefix_lm.py        A3
  models/efficient_lm.py     A4
  trainer.py           shared loop + distillation
  generation.py        decoding + submission.zip
  metrics.py           checks + BERTScore/ROUGE/BLEU
```

Tests that protect the results

`tests/test_models.py` checks each architecture:

- logits have the right shape for every prefix length;
- **causality**: changing a future token never changes an earlier prediction;
- **cache correctness**: cached generation reproduces the full forward pass.

The last test verifies the KV-cache, RoPE offsets, ALiBi offsets and sliding windows.

Pitfalls to watch for

Submission

- `prediction.json` inside a folder in the zip.
- The prompt copied into the output.
- Missing or extra IDs.
- Unchanged starter code, which does not count.

Evaluation

- Leaking training text into validation. We split by document.
- Comparing per-token losses across tokenizers. Use BERTScore or bits per word.

Training

- Only reading the start of each 10 GB file, which biases the data. This is why documents are skipped at random.
- fp16 overflow on a T4. The GradScaler handles this; watch the gradient norm.
- Losing a session without a checkpoint.
- Going over 300M parameters. The trainer refuses to start.

The viva

Be able to explain *every* line: masks, positional schemes, the loss, and each decoding knob.

Summary

- The capstone builds a **self-hosted legal foundation model** and adapts it through seven legal NLP phases.
- Phase 1 trains 50–300M-parameter models **from scratch** to continue 512-word legal prompts with 128 words, scored by BERTScore.
- We compare four substantially different designs under one controlled setup:

A1 GPT	A2 Enc–Dec	A3 Prefix-LM	A4 Efficient
causal, learned pos	cross-attention	bidirectional prompt, ALiBi	GQA, SWA, RoPE, SwiGLU

- One interface, one trainer, one decoder and one local evaluator make every experiment a single command, with its evidence saved automatically.
- The next steps are to run all four at full budget, choose decoding on the dev set, submit at least four distinct runs, and carry the best models into Phase 2.

References

- Joshi et al. (2024). IL-TUR: Benchmark for Indian Legal Text Understanding and Reasoning. arXiv:2407.05399.
- Vaswani et al. (2017). Attention Is All You Need.
- Radford et al. (2019). Language Models are Unsupervised Multitask Learners (GPT-2).
- Raffel et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5).
- Dong et al. (2019). Unified Language Model Pre-training (UniLM). Du et al. (2022). GLM.
- Press, Smith and Lewis (2022). Train Short, Test Long: Attention with Linear Biases (ALiBi).
- Su et al. (2021). RoFormer: Rotary Position Embedding.
- Touvron et al. (2023). LLaMA. Jiang et al. (2023). Mistral 7B. Gemma Team (2024). Gemma 2.
- Ainslie et al. (2023). GQA: Training Generalized Multi-Query Transformer Models.
- Shazeer (2020). GLU Variants Improve Transformer. Zhang and Sennrich (2019). RMSNorm.
- Sennrich, Haddow and Birch (2016). Neural Machine Translation of Rare Words with Subword Units (BPE).
- Holtzman et al. (2020). The Curious Case of Neural Text Degeneration (top-p).
- Zhang et al. (2020). BERTScore. Lin (2004). ROUGE. Papineni et al. (2002). BLEU.
- Hinton, Vinyals and Dean (2015). Distilling the Knowledge in a Neural Network.
- Karpathy. nanoGPT (training-loop and token-bin design).